

Lab 11- Runge-Kutta Methods

Name: Lilith Janevska

Problem 1

```
func = @(t,y) y
t0 = 0;
tn = 4;
h = 1/4;
y0 = 1;

Points = MyEulerMethod(func,y0,t0,tn,h)
Points2 = MyRK2(func,y0,t0,tn,h)
Points3 = MyRK4(func,y0,t0,tn,h)

hold on
y = exp([t0:h:tn]);
plot(transpose([t0:h:tn]),y,DisplayName="True Answer")
plot(transpose([t0:h:tn]),Points,DisplayName="Euler")
plot(transpose([t0:h:tn]),Points2,DisplayName="RK2")
plot(transpose([t0:h:tn]),Points3,DisplayName="RK4")

legend
hold off

function [result] = MyEulerMethod(f,y0,t0,tn,h)
    yn = [y0]
    i = t0;
    while i ~= tn
        yn = [yn; yn(end,1) + h.*f(i,yn(end,1))];
        i = i + h;
    end

    result = yn;
end

function [result] = MyRK2(f,y0,t0,tn,h)
    yn = [y0]
    i = t0;
    while i ~= tn
        v1 = f(i,yn(end,1));
        v2 = f(i+h,yn(end,1)+h.*v1);
        yn = [yn; yn(end,1) + (h/2).*(v1+v2)];
        i = i + h;
    end

    result = yn;
end
```

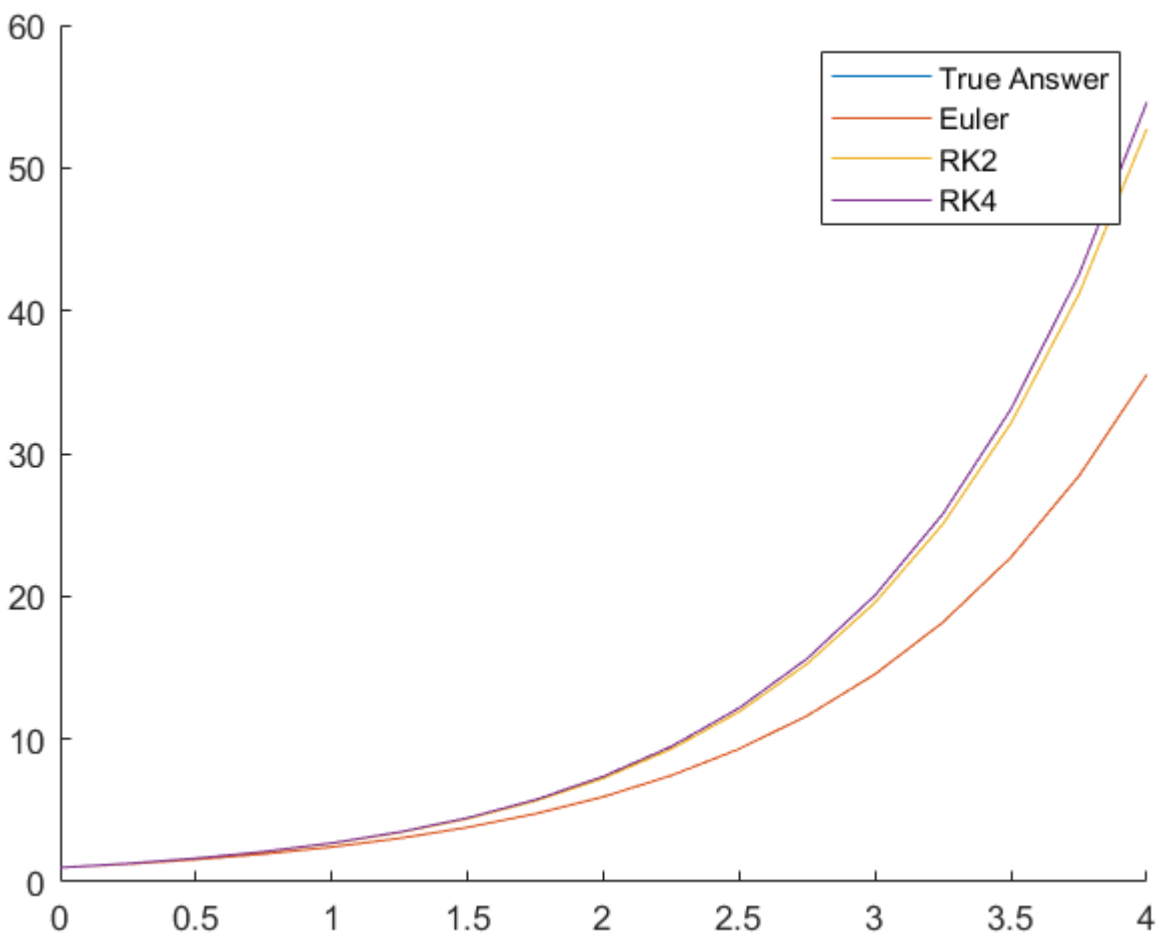
```

function [result] = MyRK4(f,y0,t0,tn,h)
    yn = [y0]
    i = t0;
    while i ~= tn
        v1 = f(i,yn(end,1));
        v2 = f(i+h/2,yn(end,1)+(h/2).*v1);
        v3 = f(i+h/2,yn(end,1)+(h/2).*v2);
        v4 = f(i+h,yn(end,1)+h.*v3);
        yn = [yn; yn(end,1) + (h/6).*(v1+2.*v2+2.*v3+v4)];
        i = i + h;
    end

    result = yn;
end

```

And we can see a nice graph for how good each method does:



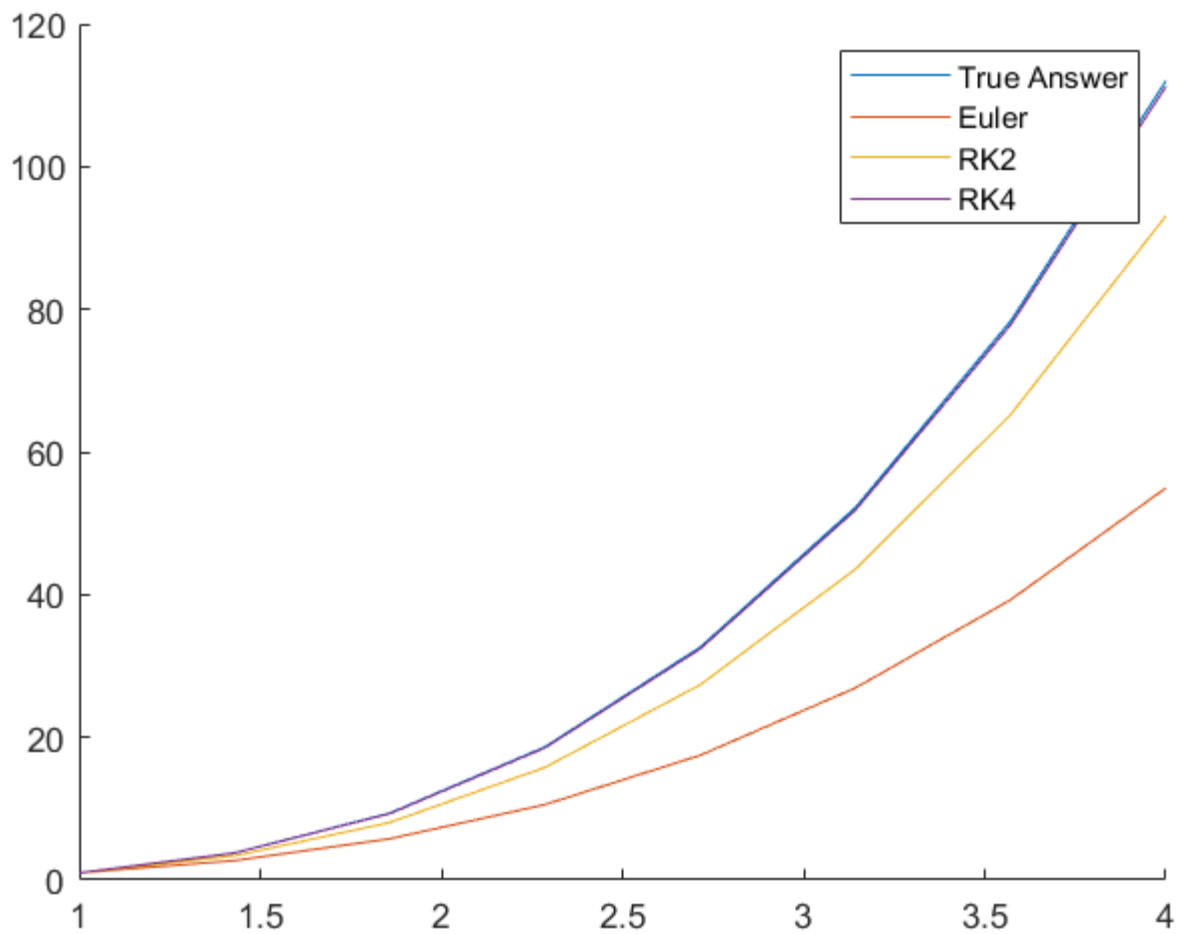
Problem 2

(a)

8×4 table

x	EulerError	RK2Error	RK4Error
1	0	0	0
1.4286	0.28385	0.10696	0.0062739
1.8571	0.38371	0.14011	0.0071686
2.2857	0.43408	0.15432	0.0072469
2.7143	0.46428	0.16142	0.0071635
3.1429	0.48436	0.16528	0.0070549
3.5714	0.49865	0.16748	0.0069531
4	0.50934	0.16878	0.0068647

(b)



Problem 3

(a)

10×4 table

n	EulerError	RK2Error	RK4Error
2	0	0	0
4	1.9808	0.45477	0.0078071
8	0.50795	0.15132	0.012445
16	0.52535	0.18158	0.0049502
32	0.51929	0.20982	0.16893
64	0.51169	0.22816	0.096148
128	0.50643	0.2386	0.072147
256	0.50337	0.24417	0.066402
512	0.50173	0.24705	0.064818
1024	0.50087	0.24852	0.062739

(b)

Yep, those seem to converge at the rates expected.

Problem 4

(a)

10×4 table

n	EulerError	RK2Error	RK4Error
2	0	0	0
4	0.30308	0.053077	0.0082835
8	0.40486	0.2932	0.28064
16	0.44896	0.32707	0.3188
32	0.47078	0.34218	0.33659
64	0.48246	0.34902	0.34517
128	0.48907	0.35206	0.34939
256	0.49298	0.35335	0.35148
512	0.49538	0.35383	0.35252
1024	0.49691	0.35396	0.35304

Here the function has an asymptote in its first derivative, and thus bot RK methods perform poorly as they have high enough percision to be affected by it, but Euler, with its lackluster percision performs as normal.

(b)

10×4 table

n	EulerError	RK2Error	RK4Error
2	0	0	0

4	0.50179	0.081544	0.00010929
8	0.4685	0.18285	0.056085
16	0.48247	0.21749	0.072828
32	0.49101	0.23403	0.081223
64	0.49547	0.24209	0.085272
128	0.49773	0.24607	0.087167
256	0.49886	0.24804	0.088041
512	0.49943	0.24902	0.089364
1024	0.49972	0.24951	0.087858

The exact same scenario occurs, but now the function's asymptote is within more derivatives, and the percision of RK2 doesn't go that far, so it performs normally. But, RK4's percision does go that far, and so performs suboptimally.

(c)

10x4 table

n	EulerError	RK2Error	RK4Error
2	0	0	0
4	0.69125	0.22542	0.00093832
8	0.50762	0.18617	0.033486
16	0.49637	0.21829	0.047325
32	0.497	0.23425	0.054779
64	0.49826	0.24215	0.058611
128	0.49907	0.24609	0.060548
256	0.49952	0.24804	0.061478
512	0.49976	0.24902	0.059199
1024	0.49988	0.24951	0.062795

Finally, the asymptote does not affect any of the methods as none of them have a large enough percision to be affected by it. All perform as normal.